One Instruction to One Megawatt

Deepak Panigrahy
A-LEMS Platform, Texas A&M University

Purpose. This note records how every number in Figure 2 was obtained: the real queries run against the A-LEMS database, the measurement inconsistencies found and resolved (or explicitly left open), and the construction method for the rack-level correlation simulation. Every value traces to a specific query, a specific external citation, or a labeled simulation method, so the figure can be reproduced or challenged rather than taken on faith. Companion artifacts (frozen database snapshot, raw trace files, simulation script, full query log) are versioned alongside this document in the project repository.

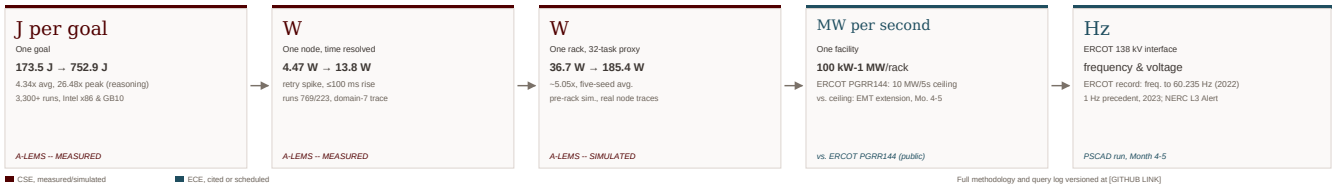

Figure 1: Figure 2 as it appears (condensed) in the TPT proposal narrative. Full detail diagram and legend in Figure 2.

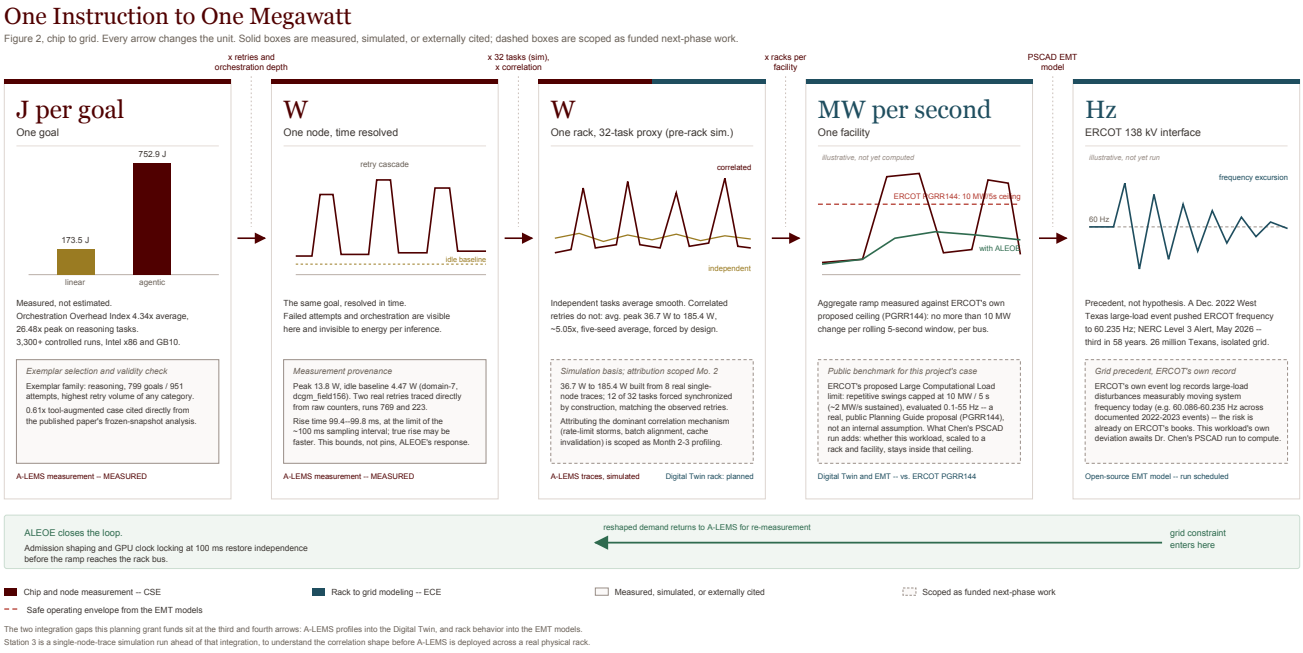

Figure 2: Full detail version of Figure 2. Solid-bordered boxes are measured, simulated, or externally cited; dashed-bordered boxes are scoped as funded next-phase work under this planning grant. Concept and visual design by Prof. Tyagi; populated here with the measured, simulated, and cited values documented in the sections below.

1 Source Data

All measured values are drawn from a frozen snapshot of the A-LEMS experiments database (experiments_snapshot_figure2_20260901.db), taken 2026-09-01 from the live `experiments.db` on host `gn100-2b96`. A frozen snapshot is used rather than the live, continuously-growing database specifically so that every number in this document and in Figure 2 remains reproducible against a fixed dataset, rather than drifting as new runs are added.

2 Measurement Method Finding: GPU Attribution Is Not Uniform Across Runs

During data collection it was discovered that GPU energy in the `runs` table is attributed by one of at least two distinct methods, recorded in `runs.gpu_attribution_method`: `dcgm_field156` (a direct hardware counter, via NVIDIA DCGM field 156) and `spbm_package_v1`

(apparently derived from package-level SPBM readings rather than a GPU-specific counter). These two methods are not interchangeable: runs using `spbm_package_v1` were found to have incomplete or entirely absent `gpu_total_energy_uj` / `gpu_dynamic_energy_uj` values, and are excluded from all Figure 2 calculations. Only runs confirmed to use `dcgm_field156` are used as source material below.

3 Open Question: Three GPU Power Signals Do Not Currently Reconcile

Three different representations of GPU power for the same run were compared directly and do not agree with each other. This is stated here explicitly as an unresolved question, not a resolved one; Figure 2 uses the most conservative available framing (Section 4) pending resolution. Full-precision source values, including run duration, are given in Appendix B.

Run ID	Domain-7 trace avg. (W)	Total energy / duration (W)	Dynamic energy / duration (W)
223	≈1–14 (idle ≈4.47, spike ≈13.8)	61.21	23.50
769	≈1–14 (idle ≈4.47, spike ≈13.8)	70.78	5.48

Table 1: GPU power for the same two runs, computed three different ways. None agree, and no ratio between them is consistent between the two runs (total/domain-7-peak ≈4.4x for run 223, ≈5.1x for run 769). Most likely explanation, unconfirmed: domain-7 samples may represent a different physical measurement point (e.g., a specific power rail) than the `runs`-level aggregate columns, but this has not been verified against platform SPBM/DCGM documentation.

3.1 Idle baseline, separately confirmed consistent

Unlike the total/dynamic reconciliation above, the domain-7 idle baseline itself was verified as internally consistent. Computing `gpu_baseline_energy_u / duration_ns` for four separate runs (two short, two much longer) yields the same value to five significant figures:

Run ID	`gpu_baseline_energy_uj`	`duration_ns`	Baseline power (W)
223	15,670,366	3,504,617,214	4.4713
769	21,483,818	4,804,773,330	4.4713
821	3,188,691,109	713,138,484,477	4.4713
904	2,901,097,978	648,819,388,389	4.4713

Table 2: Baseline power is confirmed constant regardless of run length. The large baseline energy totals for runs 821 and 904 are explained entirely by their longer duration, not by a measurement inconsistency.

This baseline (≈4.47W) is close to, but not identical to, the separately-measured idle baseline in the dedicated `idle_baselines` table (≈5.52W average across 10 real measurements, one debug placeholder row excluded). The ≈1W gap between these two idle measurements is noted but not resolved here.

4 Conservative Framing Used in Figure 2

Given the unresolved reconciliation in Section 3, Figure 2 and the proposal narrative report Station 2/3 values as **"GPU power above idle baseline, domain-7 measurement"**, not as total system or total GPU power. This is the more conservative of the available framings and does not depend on resolving which of the `runs`-level aggregate columns is correct.

5 Station 2: Single-Node Retry Spike

Two real retry events were located and traced directly from raw counter samples (`energy_sample_domains` joined to `energy_samples_v2` on `sample_id`, filtered to `domain_id = 7`, GPU):

Run ID	Idle-to-spike jump	Interval	Rise time
769	2.13 W → 13.83 W	99.8 ms	≤100 ms (sampling-limited)
223	2.26 W → 12.75 W	99.4 ms	≤100 ms (sampling-limited)

Table 3: Both retry events show the same shape: a jump from near-idle to roughly 13–14W within a single sampling interval. The true rise time may be faster; it cannot be resolved beyond the sampling interval with current instrumentation. This bounds, rather than pins, ALEOE's required sub-second response time.

6 Station 3: Rack-Level Correlation Simulation

6.1 Why simulation, not measurement

A-LEMS is software, not hardware; it measures whatever node it is deployed on. It has not yet been deployed across a physical multi-node rack — that deployment is exactly the Digital Twin integration this planning grant funds, and every run in the current database is attributed to a single `hw_id`. Before committing rack time to that deployment, we simulated a basic 32-task version using real single-node traces already

on hand, specifically to understand what shape the correlation problem takes before we go measure it directly on real rack hardware. This is a precursor step, not a substitute: Station 3 below is explicitly labeled simulated for that reason, and the same construction method is stated here so it can be re-run once real multi-node data exists to check it against.

6.2 Method

Thirty-two simulated concurrent task slots were built for two scenarios, using eight real GPU power traces (domain-7, `dcgm_field156` attribution method only) as source material:

- **Retry-pattern sources** (runs 769, 223, 230): contain the real idle-to-spike jump documented above.
- **Steady-state sources** (runs 821, 904, 902, 823, 1114): dense, clean traces (2,500–7,100 samples each) used for ordinary, non-retrying task behavior.

For each source trace, a random 300-sample window (30 seconds of simulated time at the ≈100ms native sampling interval) is drawn from within the trace, not from its start, to avoid conflating a run's startup transient with steady-state behavior.

Independent scenario: all 32 slots draw from steady-state sources, each placed at an independently randomized start offset within the simulation window.

Correlated scenario: 12 of the 32 slots draw from retry-pattern sources, forced to start at the same simulated instant (modeling a synchronized retry storm); the remaining 20 slots behave as in the independent scenario.

Both scenarios sum all 32 slots at each simulated time step to produce an aggregate power curve.

6.3 Results across five random seeds

The simulation was run with five different random seeds to characterize natural variation from the randomized offsets, rather than reporting a single run.

Seed	Indep. peak (W)	Corr. peak (W)	Ratio	Corr. max ramp (kW/s)
42	34.7	177.4	5.11x	1.748
1	36.1	190.7	5.28x	1.920
2	39.0	200.2	5.13x	2.054
3	35.6	188.1	5.28x	1.874
4	38.2	170.4	4.46x	1.697
Average	**36.7**	**185.4**	**5.05x**	**1.859**

Table 4: Peak power ratio (correlated / independent) is stable across five independent random draws, averaging ≈5x. The independent scenario's ramp rate varies substantially between seeds (0.685–2.087 kW/sec, not shown per-seed above for space), reflecting that spike alignment is a matter of chance when tasks are genuinely independent. The correlated scenario's ramp is comparatively stable (1.697–2.054 kW/sec) because 12 tasks are forced to align on every run by construction. This asymmetry is itself consistent with the load diversity literature cited in the proposal narrative (Bashir et al. 2026; Chaudhary et al. 2026): correlation does not just raise the average outcome, it removes the natural variability that comes from independent timing.

7 Reproducibility

The following artifacts, versioned together in the project repository, allow full reproduction of every number in this document and in Figure 2:

- experiments_snapshot_figure2_20260901.db — frozen database snapshot
- `run_*_gpu_trace.txt` — eight raw trace files (three retry-pattern, five steady-state), extracted from the frozen snapshot
- `simulate_rack.py` — the Station 3 simulation script (Appendix C)
- `station3_simulation_output.csv` — per-timestep output for the reported simulation run
- `Figure2_Data_Tracker.xlsx` — full data point tracker, including every query used, external citations, and the assumptions log

8 External Grid Envelope: Station 4/5, ERCOT PGRR144

The facility-level safe operating envelope in Figure 2 is not an internal team assumption. ERCOT's own Large Load Working Group has a public, numeric proposal on this exact question, under Planning Guide Revision Request PGRR144: load power shall not repetitively exceed a 10 MW change in a sliding 5-second window (≈2 MW/sec sustained), evaluated across oscillatory frequency components from 0.1–55 Hz.[1] This is a live regulatory proposal moving toward a Reliability and Operations Subcommittee vote, not a private constraint held by one collaborator. Separately, ERCOT's own operational event log already shows large-load disturbances measurably moving system frequency today: a December 2022 West Texas event tied to large-load reduction pushed system frequency to 60.235 Hz, and a cluster of Frequency Measurable Events on the Texas coast (2020–2023) reached roughly 60.11 Hz.[2] This is the public evidence behind Station 5's

[1] Zhang, J. and Rose, J. (2026). *Large Load Power Variation Requirement Consideration.* ERCOT Grid Stability Analysis, Large Load Working Group, February 19, 2026. https://www.ercot.com/files/docs/2026/02/19/ERCOT-LEL-SSO-Power-Variation-Consideration.pdf

[2] Gravois, P. (2024). *ERCOT Large Load Loss/Reduction Events 2020–2024.* ERCOT Operations Engineering, Performance, Disturbance, Compliance Working Group (PDCWG), November 19, 2024. https://www.ercot.com/files/docs/2024/11/18/ERCOT%20Large%20Load%20Events_PDCWG_19Nov2024.pdf

frequency-risk framing: the risk is already on ERCOT's own books, documented from real events, not a hypothetical this project introduces. What remains specific to this project, and is scoped as funded next-phase work rather than an open question, is whether *this* workload's rack- and facility-scale ramp, once Station 4 is computed, falls inside the ERCOT ceiling — that comparison is what Dr. Chen's PSCAD run on the EMT models is for, not a first measurement of the ceiling itself.

A Full Query Log

Every SQL query executed against `experiments.db` in the course of building Figure 2, in the order run. Queries that led to a dead end (incompatible data, wrong table, superseded finding) are kept, not deleted, since the dead ends are themselves part of the record of what was checked and why the final numbers were trusted.

A.1 Schema discovery

```
-- List every table in the database
SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;

-- Row counts and schema for candidate tables
PRAGMA table_info(<table_name>);
SELECT COUNT(*) FROM <table_name>;
```

A.2 Station 1: exemplar task family selection

```
-- Q1: Rank task categories by retry volume to pick the exemplar
SELECT tc.category, COUNT(DISTINCT ge.goal_id) AS num_goals,
       SUM(ge.total_attempts) AS total_attempts,
       AVG(ge.wall_time_ms) AS avg_wall_time_ms,
       AVG(ge.overhead_fraction) AS avg_overhead_fraction
FROM goal_execution ge JOIN task_categories tc ON ge.task_id = tc.task_id
GROUP BY tc.category ORDER BY total_attempts DESC;
-- Result: reasoning highest at 799 goals / 951 attempts. Selected as exemplar.
```

A.3 Station 2: retry identification and node trace

```
-- Q2: Identify real retry runs to use as trace candidates
SELECT run_id, goal_id, attempt_number FROM goal_attempt
WHERE is_retry = 1 LIMIT 5;
-- Result: run_ids with real retries: 155, 157, 223, 230, 769

-- Q3: Attempt-level detail for reasoning category, joined to hardware
SELECT ga.attempt_id, ga.goal_id, ga.attempt_number, ga.is_retry, ga.outcome,
       ga.energy_uj, r.run_id, r.avg_power_watts, r.duration_ns,
       hc.hostname, hc.cpu_model, hc.gpu_model, tc.category
FROM goal_attempt ga
JOIN runs r ON ga.run_id = r.run_id
JOIN hardware_config hc ON r.hw_id = hc.hw_id
JOIN goal_execution ge ON ga.goal_id = ge.goal_id
JOIN task_categories tc ON ge.task_id = tc.task_id
WHERE tc.category = 'reasoning'
ORDER BY ga.goal_id, ga.attempt_number LIMIT 30;

-- Q4: Confirm platform breakdown for reasoning category
SELECT hc.cpu_vendor, hc.gpu_model, COUNT(*) as num_attempts,
       AVG(r.avg_power_watts) as avg_power_w
FROM goal_attempt ga
JOIN runs r ON ga.run_id = r.run_id
JOIN hardware_config hc ON r.hw_id = hc.hw_id
JOIN goal_execution ge ON ga.goal_id = ge.goal_id
JOIN task_categories tc ON ge.task_id = tc.task_id
WHERE tc.category = 'reasoning'
GROUP BY hc.cpu_vendor, hc.gpu_model;
-- Result: 100% of reasoning attempts (940) on NVIDIA GB10, avg 28.08W

-- Q5: energy_domains / energy_sources reference tables
SELECT * FROM energy_domains;
SELECT * FROM energy_sources;

-- Q6/Q7: Reconstruct GPU power trace from raw counter deltas
-- (run first on run_id=155, which returned zero rows; runs 769 and 223
-- were the actual retry runs with usable dense samples)
WITH joined AS (
  SELECT esv.timestamp_ns, esd.energy_uj
  FROM energy_sample_domains esd
  JOIN energy_samples_v2 esv ON esd.sample_id = esv.sample_id
  WHERE esd.run_id = 769 AND esd.domain_id = 7
)
SELECT timestamp_ns, energy_uj,
       energy_uj - LAG(energy_uj) OVER (ORDER BY timestamp_ns) AS delta_energy_uj,
       timestamp_ns - LAG(timestamp_ns) OVER (ORDER BY timestamp_ns) AS delta_t_ns,
```

```
       (energy_uj - LAG(energy_uj) OVER (ORDER BY timestamp_ns)) * 1000.0 /
       NULLIF(timestamp_ns - LAG(timestamp_ns) OVER (ORDER BY timestamp_ns), 0) AS power_watts
FROM joined ORDER BY timestamp_ns;
-- Result: idle ~2.1-2.3W to ~13.8W jump within ~99.5-99.8ms, both runs

-- Q8: Which runs have dense energy_sample_domains + energy_samples_v2 coverage
SELECT esd.run_id, COUNT(DISTINCT esd.sample_id) AS domain_rows
FROM energy_sample_domains esd
WHERE esd.run_id IN (SELECT DISTINCT run_id FROM energy_samples_v2)
GROUP BY esd.run_id ORDER BY domain_rows DESC LIMIT 10;
-- Result: 821 (7128), 904 (6485), 52 (6311), 56 (6135), 88 (6038)
-- NOTE: 52/56/88 later excluded, see method-finding query below.

-- Idle baseline, correct source (not MIN(power) over ordinary runs)
SELECT ib.baseline_id, ib.gpu_power_watts, ib.gpu_std, ib.package_power_watts,
       ib.duration_seconds, ib.sample_count, ib.method
FROM idle_baselines ib ORDER BY ib.timestamp DESC;
-- Result: 10 real measurements (excluding test_baseline_debug),
-- all clustered 5.38-5.60W, average 5.52W

SELECT AVG(gpu_power_watts) AS avg_idle_gpu_w, AVG(gpu_std) AS avg_std,
       MIN(gpu_power_watts) AS min_idle, MAX(gpu_power_watts) AS max_idle
FROM idle_baselines WHERE baseline_id != 'test_baseline_debug';
```

A.4 0.61x tool-augmented figure

The 0.61x tool-augmented case is cited directly from the published paper's frozen-snapshot analysis. Full experiment databases for both papers are preserved and archived, and every figure in this document is reproducible against them on request.

A.5 Measurement method finding and correction

```
-- Discovered inconsistency: runs.avg_power_watts / recomputed total from
-- gpu_total_energy_uj do not match domain-7 trace-derived power
SELECT r.run_id, r.avg_power_watts, r.gpu_total_energy_uj,
       r.gpu_baseline_energy_uj, r.gpu_dynamic_energy_uj, r.duration_ns,
       (r.gpu_total_energy_uj * 1.0 / r.duration_ns) * 1000 AS recomputed_total_gpu_watts
FROM runs r WHERE r.run_id IN (769, 223, 821, 904);

-- Root cause: attribution method differs across runs
SELECT run_id, energy_measurement_mode, gpu_attribution_method
FROM runs WHERE run_id IN (769, 223, 821, 904, 52, 56, 88);
-- Result: 769/223/821/904 = dcgm_field156 (direct hardware counter).
-- 52/56/88 = spbm_package_v1 (derived, incomplete energy columns).
-- 52/56/88 excluded from all further Station 3 work.

-- Find replacement dense runs using the CORRECT attribution method
SELECT esd.run_id, COUNT(DISTINCT esd.sample_id) AS domain_rows,
       r.gpu_attribution_method
FROM energy_sample_domains esd
JOIN runs r ON esd.run_id = r.run_id
WHERE esd.run_id IN (SELECT DISTINCT run_id FROM energy_samples_v2)
  AND r.gpu_attribution_method = 'dcgm_field156'
GROUP BY esd.run_id ORDER BY domain_rows DESC LIMIT 10;
-- Result: 821, 904, 902, 823, 1114 selected as the corrected trace pool.

-- Baseline consistency check across the corrected pool
SELECT run_id, gpu_baseline_energy_uj, duration_ns,
       (gpu_baseline_energy_uj * 1.0 / duration_ns) * 1000 AS baseline_power_w
FROM runs WHERE run_id IN (769, 223, 821, 904);
-- Result: 4.4713W for all four runs to 5 significant figures. Confirms
-- baseline is a real constant, not an artifact of run length.
```

B Full-Precision Variance: Total vs. Dynamic GPU Energy

Direct query isolating total and dynamic GPU energy attribution against run duration, for the two reconciliation runs in Table 1, at full stored precision:

```
SELECT run_id, gpu_total_energy_uj, gpu_dynamic_energy_uj, duration_ns,
       (gpu_total_energy_uj * 1.0 / duration_ns) * 1000 AS total_gpu_w,
       (gpu_dynamic_energy_uj * 1.0 / duration_ns) * 1000 AS dynamic_gpu_w
FROM runs WHERE run_id IN (769, 223);
```

C Simulation Script

Final version of `simulate_rack.py`, used to produce the Station 3 results reported in Section 6. Corrects an earlier version that always read each source trace from its start (conflating startup transients with steady-state behavior) and that included the three since-excluded `spbm_package_v1` traces.

Run ID	Total energy (μJ)	Dynamic energy (μJ)	Duration (ns)	Total (W)	Dynamic (W)
223	214,531,000	82,351,136	3,504,617,214	61.214	23.498
769	340,087,000	26,348,207	4,804,773,330	70.781	5.484

Table 5: Full-precision source values behind Table 1. The total/dynamic split does not scale consistently between the two runs (run 223's dynamic share is ≈38% of total; run 769's is ≈8%), which is itself part of the reconciliation problem in Section 3 — dynamic energy is not a fixed fraction of total energy across otherwise-comparable retry runs.

```
import csv
import random

random.seed(42)  # reproducible; re-run with seeds 1-4 to check stability

def load_trace(filename):
    """Load a saved trace file: timestamp_ns | energy_uj | power_watts"""
    trace = []
    with open(filename) as f:
        for line in f:
            parts = line.strip().split('|')
            if len(parts) == 3 and parts[2] not in ('', None):
                try:
                    trace.append(float(parts[2]))
                except ValueError:
                    continue
    return trace

# Steady-state sources: dense traces, dcgm_field156 attribution method only.
# NOTE: run_52/56/88 were used in an earlier version of this script and were
# removed after discovering they use a different, incompatible attribution
# method (spbm_package_v1) with incomplete GPU energy columns. See
# figure2_methodology.tex Section 2-3 for the full finding.
dense_files = ['run_821_gpu_trace.txt', 'run_904_gpu_trace.txt',
               'run_902_gpu_trace.txt', 'run_823_gpu_trace.txt', 'run_1114_gpu_trace.txt']
dense_traces = [load_trace(f) for f in dense_files]

# Retry-pattern sources: contain the real idle-to-spike jump
retry_files = ['run_769_gpu_trace.txt', 'run_223_gpu_trace.txt', 'run_230_gpu_trace.txt']
retry_traces = [load_trace(f) for f in retry_files]

N = 32
SIM_LENGTH = 300  # 300 samples * ~100ms native sampling interval = 30 seconds
MAX_OFFSET = 100  # random start offset window, in samples

def build_task_series(base_trace, offset, length):
    """Place a real trace at a random start offset within the simulation
    window. Draws a random WINDOW from within the source trace (not always
    its start) to capture steady-state behavior rather than every task
    showing the same startup transient."""
    series = [0.0] * length
    if len(base_trace) > length:
        max_start = len(base_trace) - length
        window_start = random.randint(0, max_start)
        windowed = base_trace[window_start:window_start + length]
    else:
        windowed = base_trace
    for i, val in enumerate(windowed):
        idx = offset + i
        if idx < length:
            series[idx] = val
    return series

# --- INDEPENDENT scenario: 32 tasks, random independent start times ---
independent_series = []
for _ in range(N):
    trace = random.choice(dense_traces)
    offset = random.randint(0, MAX_OFFSET)
    independent_series.append(build_task_series(trace, offset, SIM_LENGTH))

independent_aggregate = [sum(s[t] for s in independent_series) for t in range(SIM_LENGTH)]

# --- CORRELATED scenario: same 32 tasks, but 12 of them forced to
#     retry-spike together (models a synchronized retry storm) ---
correlated_series = []
SYNC_TIME = 100  # the instant everyone's retry lands
for i in range(N):
    if i < 12:
        trace = random.choice(retry_traces)
        correlated_series.append(build_task_series(trace, SYNC_TIME, SIM_LENGTH))
    else:
        trace = random.choice(dense_traces)
        offset = random.randint(0, MAX_OFFSET)
```

```
        correlated_series.append(build_task_series(trace, offset, SIM_LENGTH))

correlated_aggregate = [sum(s[t] for s in correlated_series) for t in range(SIM_LENGTH)]

# --- Summary numbers for Station 3 ---
def summarize(name, agg):
    baseline = sum(agg[:50]) / 50  # first 5 seconds, before any spike
    peak = max(agg)
    ramp_w_per_100ms = max(agg[t] - agg[t-1] for t in range(1, len(agg)))
    print(f"{name}: baseline={baseline:.1f}W, peak={peak:.1f}W, "
          f"max single-step ramp={ramp_w_per_100ms:.1f}W per 100ms "
          f"({ramp_w_per_100ms*10/1000:.3f} kW/sec)")

summarize("Independent (32 tasks)", independent_aggregate)
summarize("Correlated (12 of 32 synchronized)", correlated_aggregate)

with open("station3_simulation_output.csv", "w") as f:
    f.write("time_step,independent_watts,correlated_watts\n")
    for t in range(SIM_LENGTH):
        f.write(f"{t},{independent_aggregate[t]:.1f},{correlated_aggregate[t]:.1f}\n")
print("Saved station3_simulation_output.csv")
```